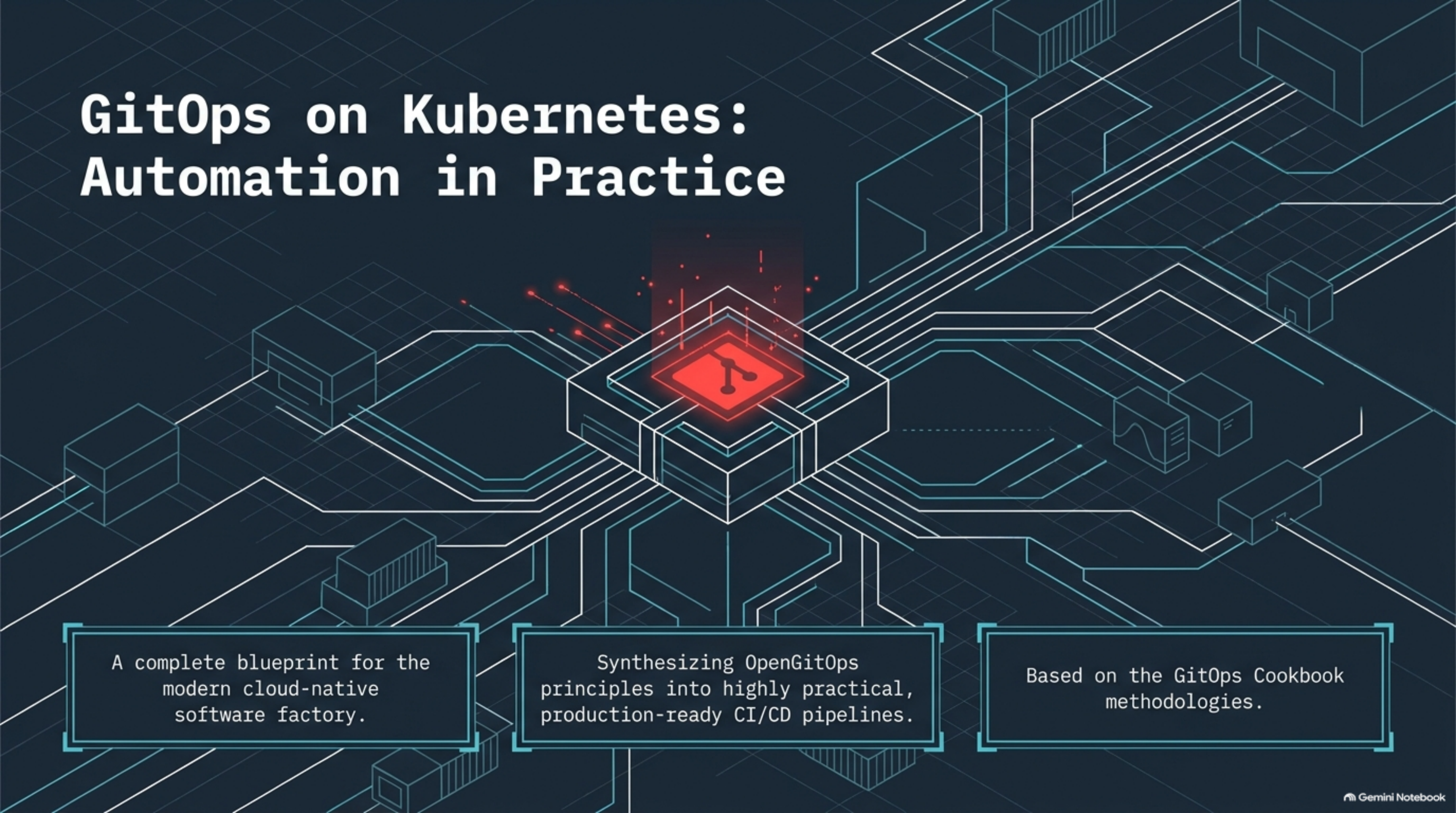


GitOps on Kubernetes: Automation in Practice

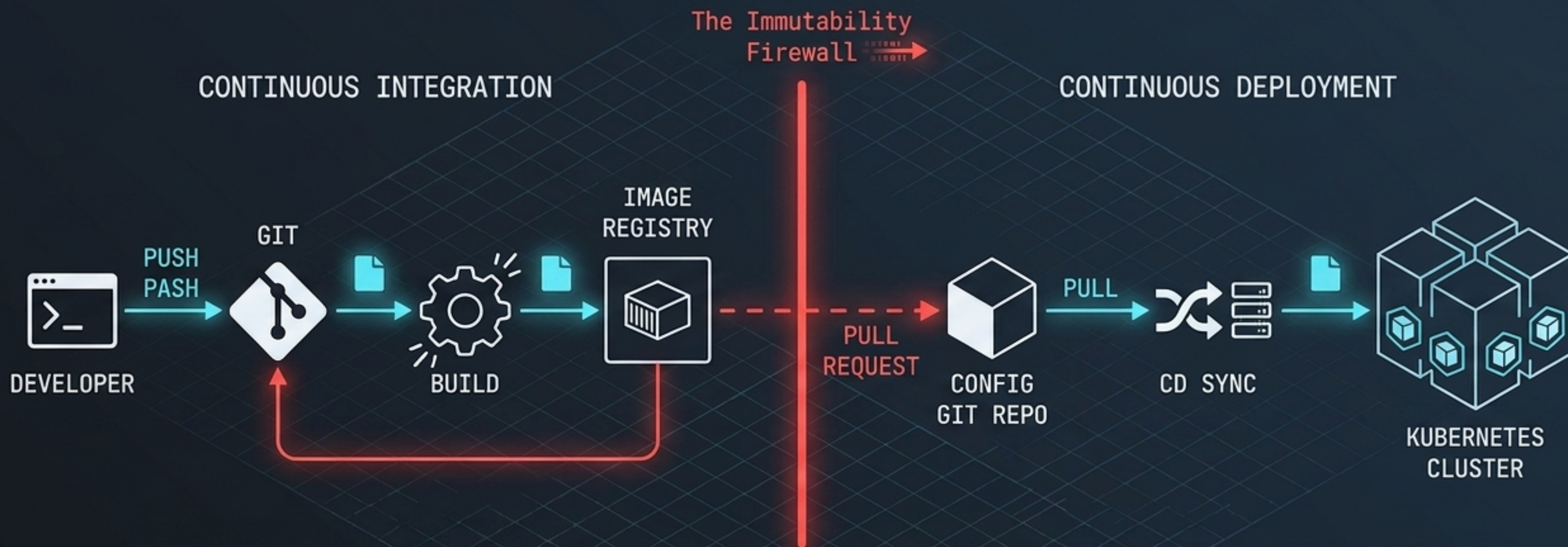


A complete blueprint for the modern cloud-native software factory.

Synthesizing OpenGitOps principles into highly practical, production-ready CI/CD pipelines.

Based on the GitOps Cookbook methodologies.

The Immutability Firewall



The Paradigm Shift:
GitOps replaces traditional push-based CI/CD with a pull-based developer workflow.

Complete Isolation:
CI builds and pushes immutable images. CD pulls declarative state.

Git as the Source of Truth:
Every operation, cluster update, or infrastructure change is only possible through a Git workflow.

The Four Pillars of OpenGitOps



Declarative

The system's desired state must be expressed declaratively.

Versioned & Immutable

State is stored in a way that enforces immutability, versioning, and retains a complete audit history.

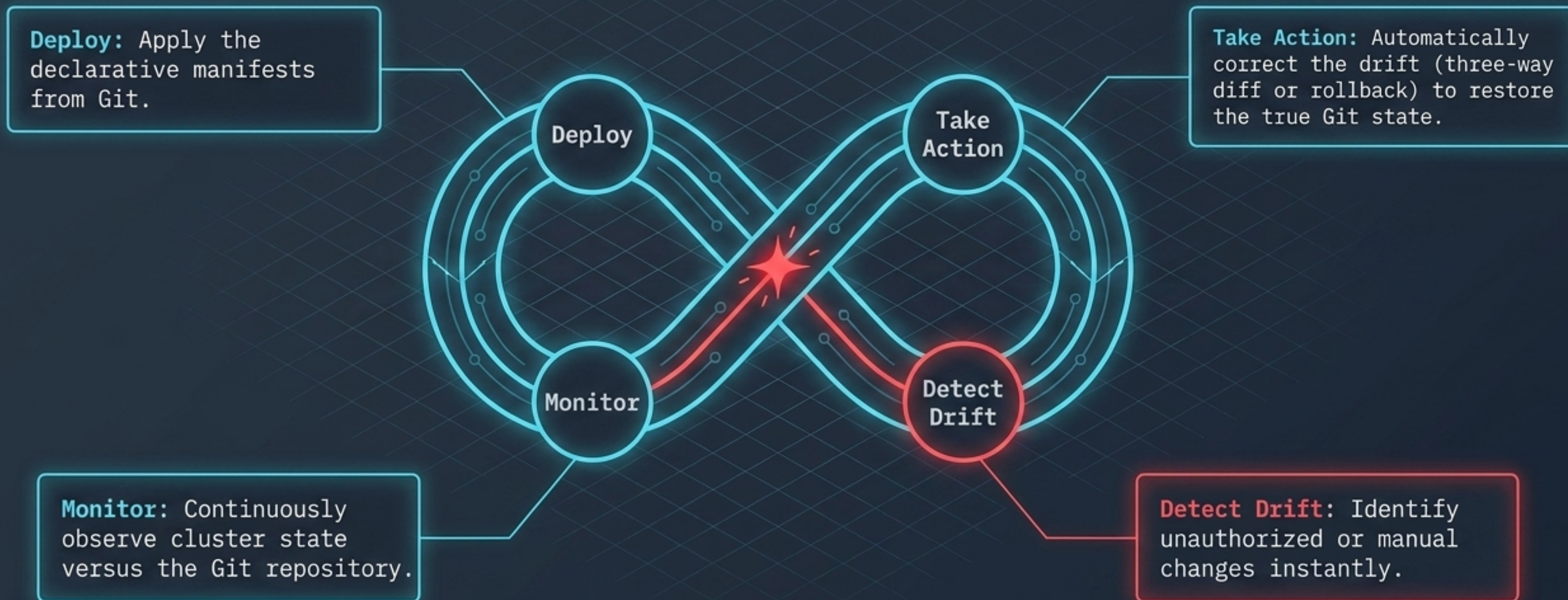
Pulled Automatically

Software agents automatically pull the desired state declarations from the source.

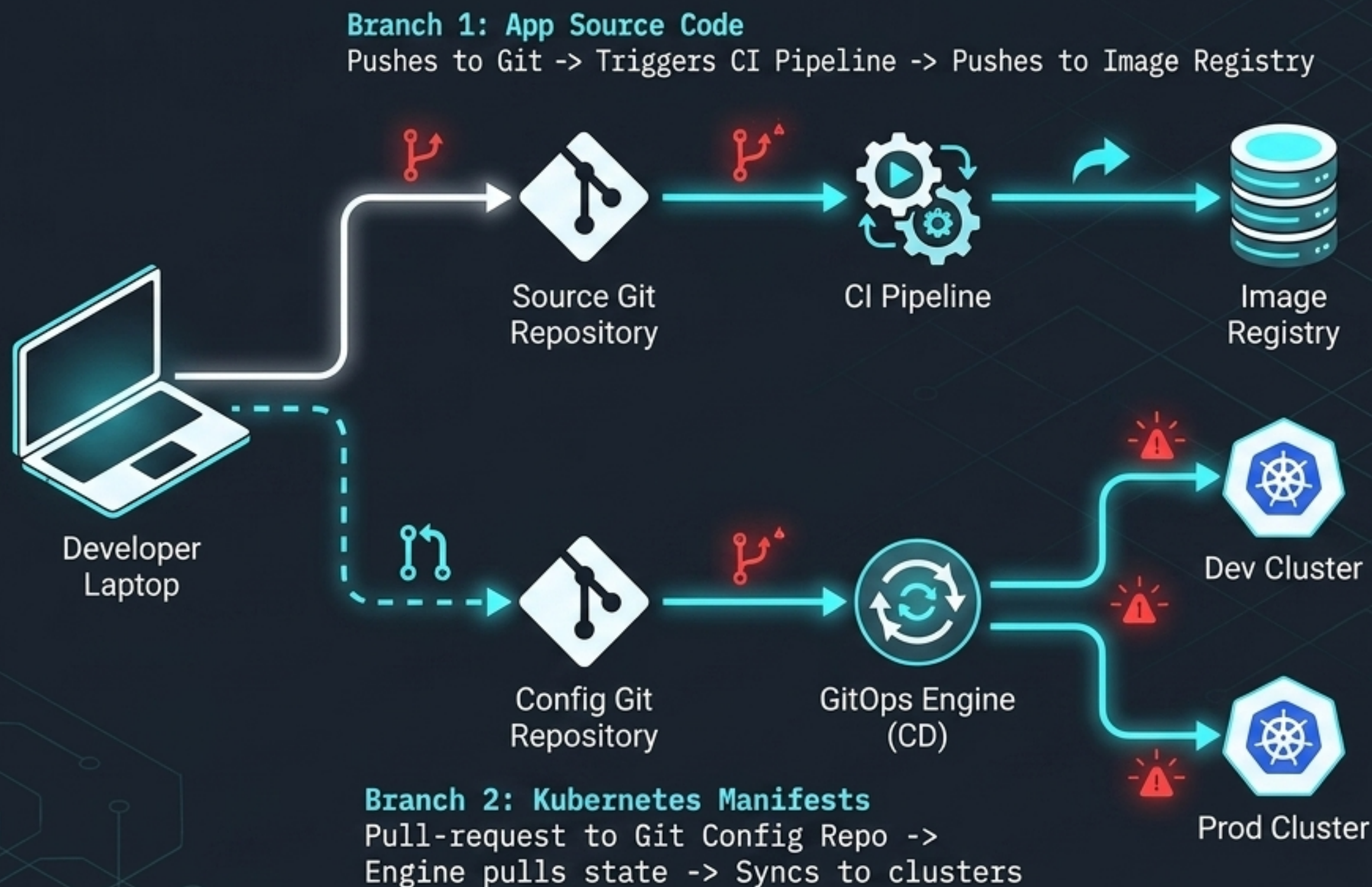
Continuously Reconciled

Agents continuously observe the actual system state and attempt to apply the desired state.

The Synchronization Loop



The Dual-Repository Architecture

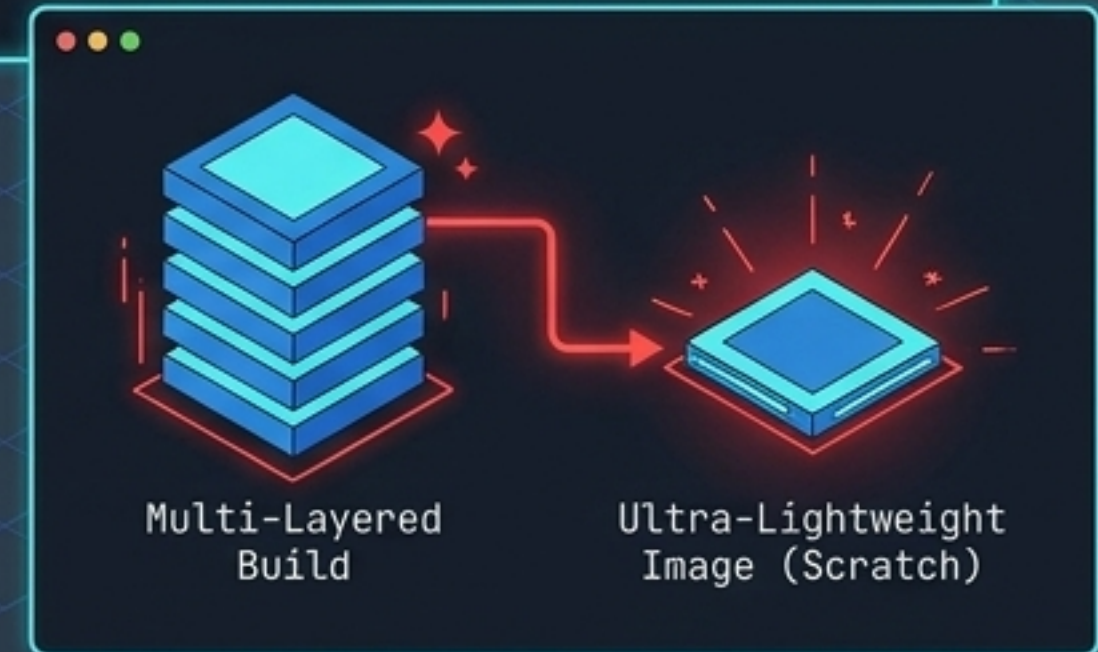


Why Separate Repositories?

- **Prevents Infinite Loops:** CI pipelines won't accidentally trigger a CD sync simply because a readme or test changed.
- **Access Control:** Developers own application code; Platform/SRE teams can lock down manifest configurations.
- **Auditability:** A clean, isolated commit history exclusively for infrastructure and deployment state.

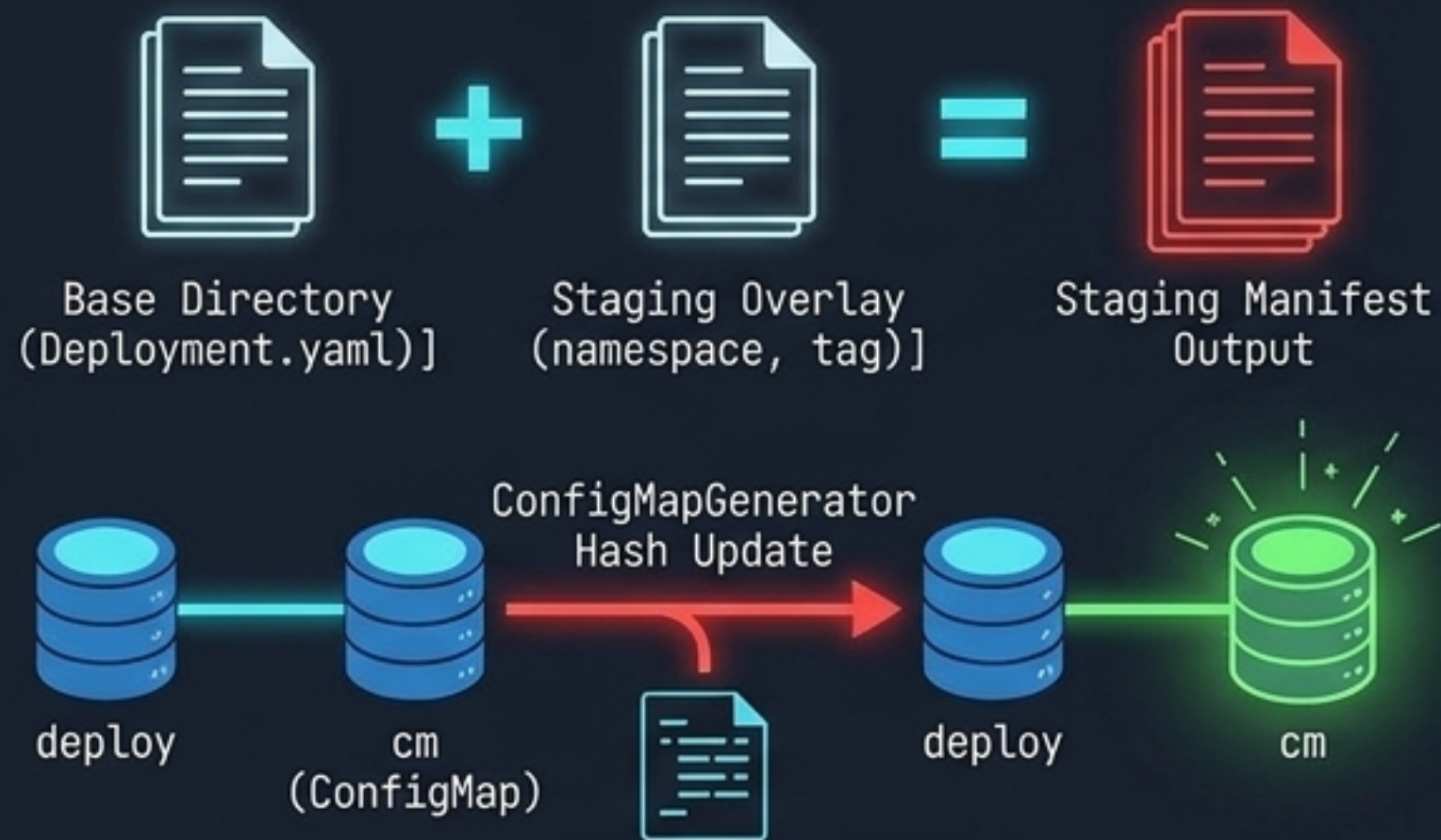
Container Packaging Diagnostic Matrix

Tool	Mechanism	Best For
Docker	Standard Dockerfile, requires privileged daemon.	Local development environments.
Jib	Pure Java, daemonless, no Dockerfile required.	High-speed Java CI/CD. Separates dependencies from classes for lightning-fast rebuilds.
Buildah	Daemonless, rootless, builds from scratch.	Highly secure CI/CD pipelines. Creates empty layers to produce ultra-lightweight images.



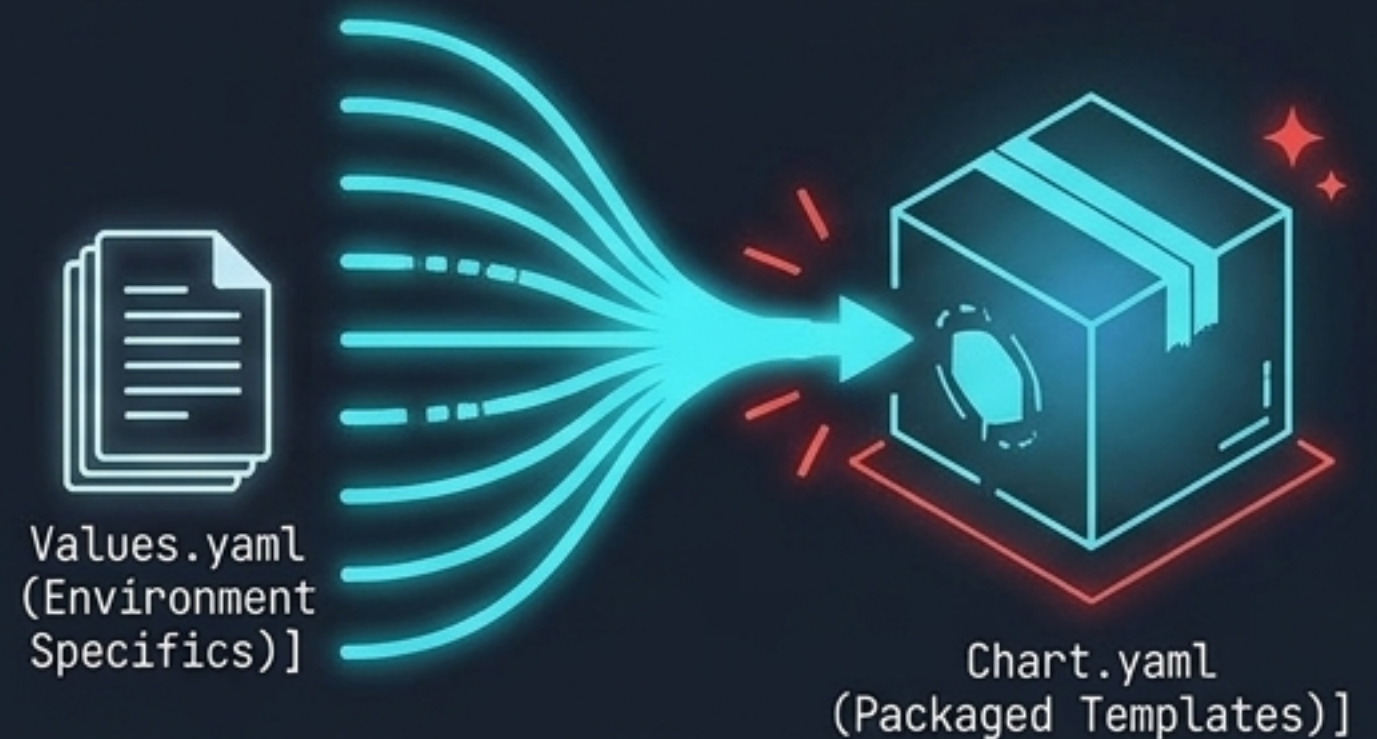
Manifest Configuration: Kustomize vs. Helm

Kustomize



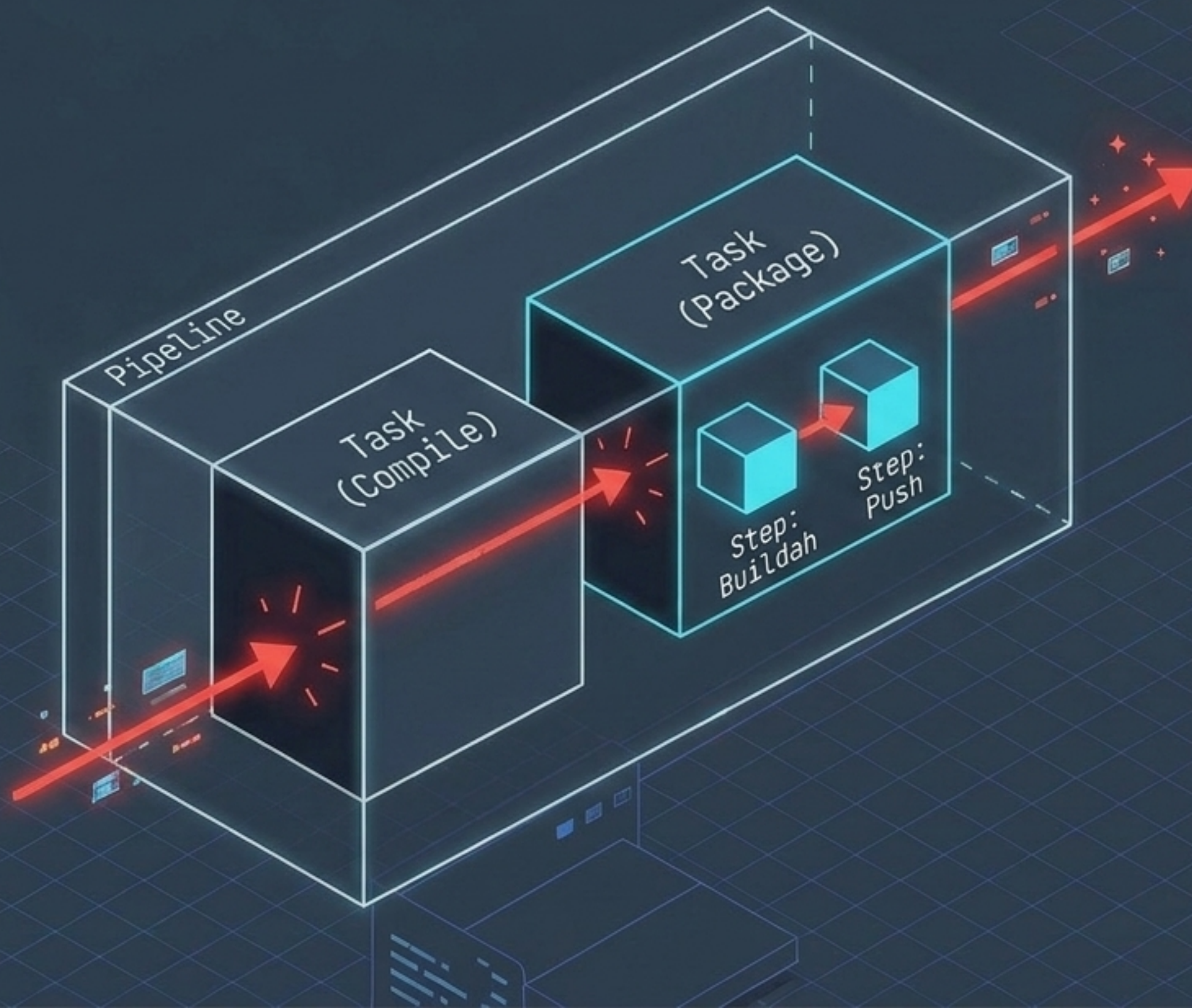
Kustomize: Template-free patching. Maintains standard Kubernetes YAML structure and uses overlays for multiple environments.

Helm



Helm: Powerful package management and templating. Best for distributing complex, configurable applications across diverse clusters.

Cloud-Native CI with Tekton



Kubernetes-Native Design: Tekton runs natively inside the cluster using Custom Resource Definitions (CRDs). No external CI servers required.

Stateless & Decoupled: Tasks are highly reusable building blocks that execute inside isolated pods.

Secure Assembly: Seamlessly executes rootless Buildah or Kaniko container builds directly within the Kubernetes environment.

The CD Engine: Argo CD



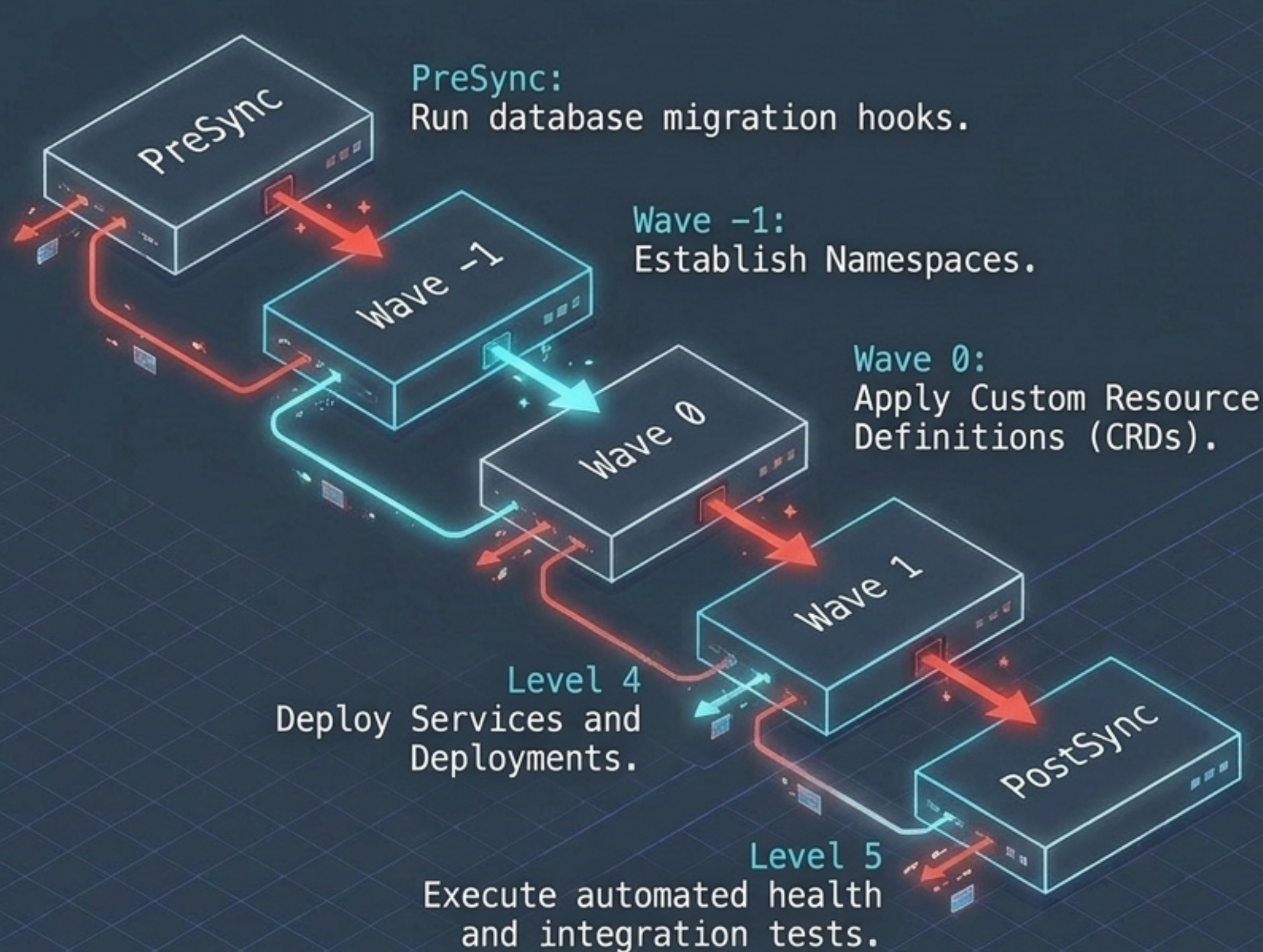
Automated Synchronization:

Eliminates the need for manual `kubectl apply` commands or exposed API credentials.

Ruthless Self-Healing:

If a manual drift is detected (e.g., an unauthorized patch modifying an environment variable), Argo CD instantly corrects the drift, reverting the cluster to the precise Git state.

Orchestrating Complexity: Argo CD Sync Waves



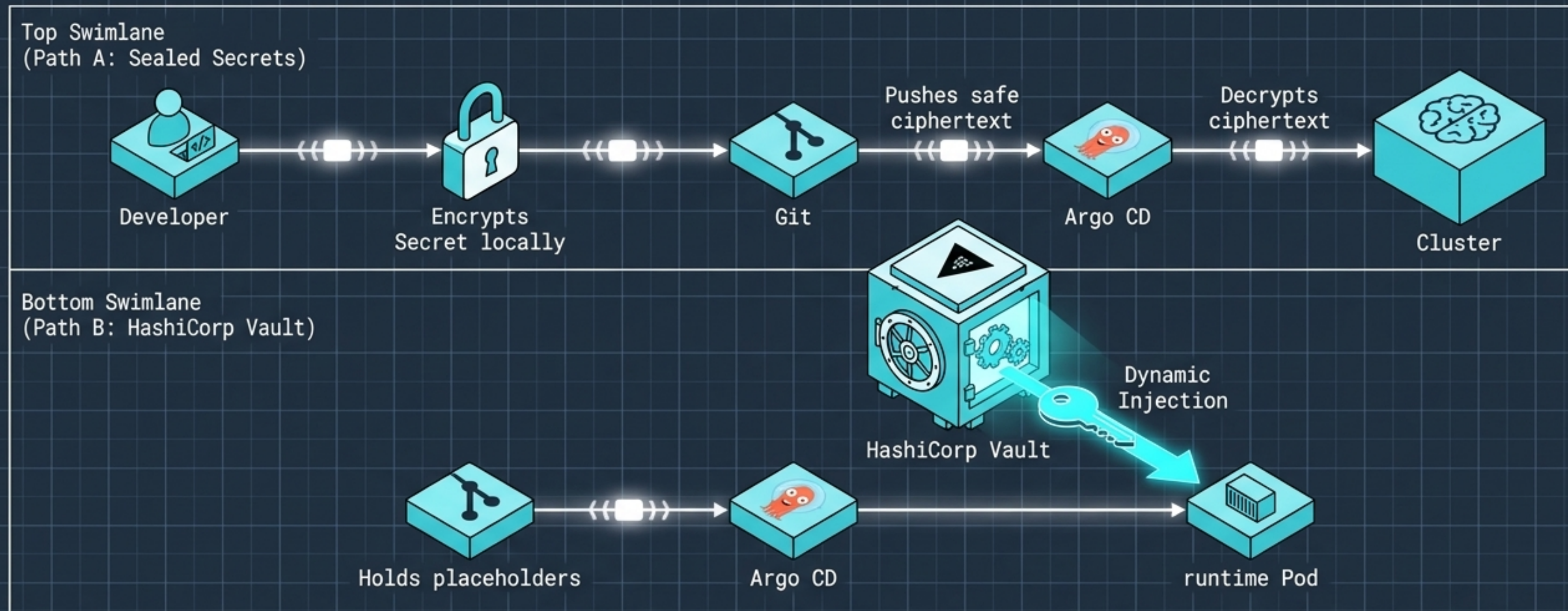
Order of Operations:

Applications rarely boot simultaneously. Sync waves prevent race conditions and failed starts.

Granular Control:

Assign specific weights and phases to Kubernetes manifests, ensuring dependencies (like a database) are fully healthy before the application spins up.

Advanced Security: Secrets as Code



The Rule:

Never store plain-text credentials in Git.

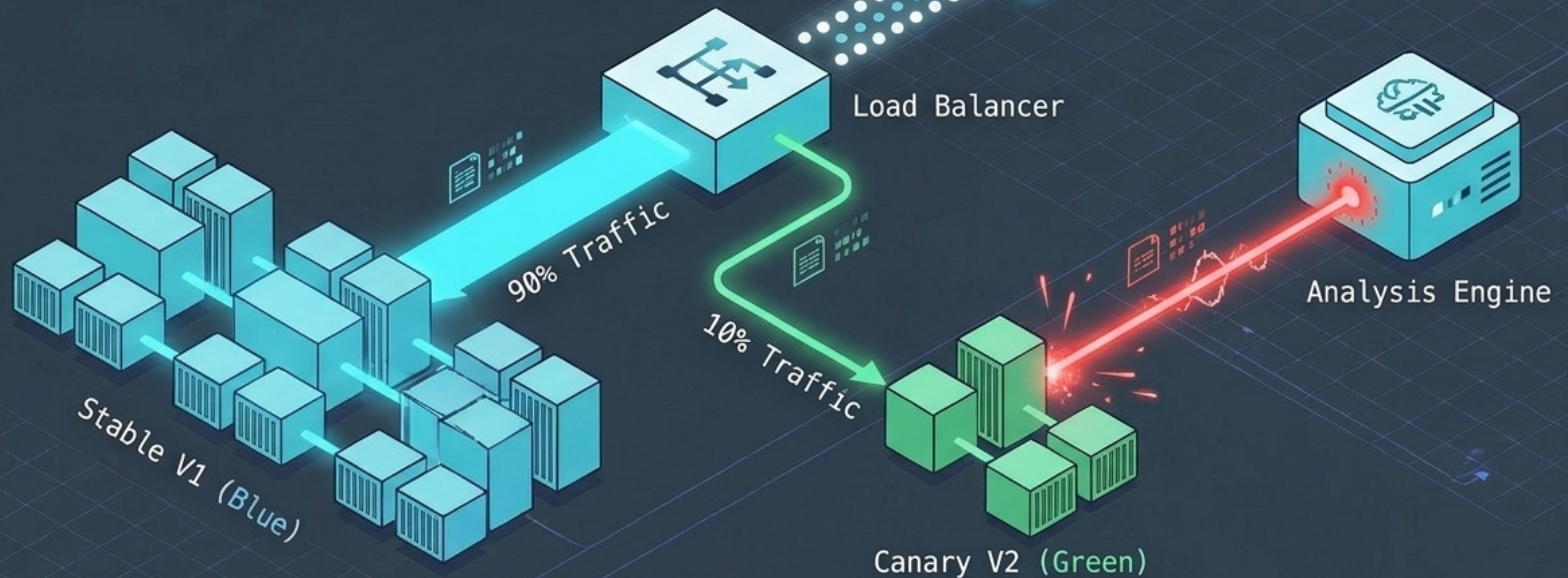
Asymmetric Encryption:

Sealed Secrets allow developers to commit encrypted configurations safely to public or private repos.

Dynamic Injection:

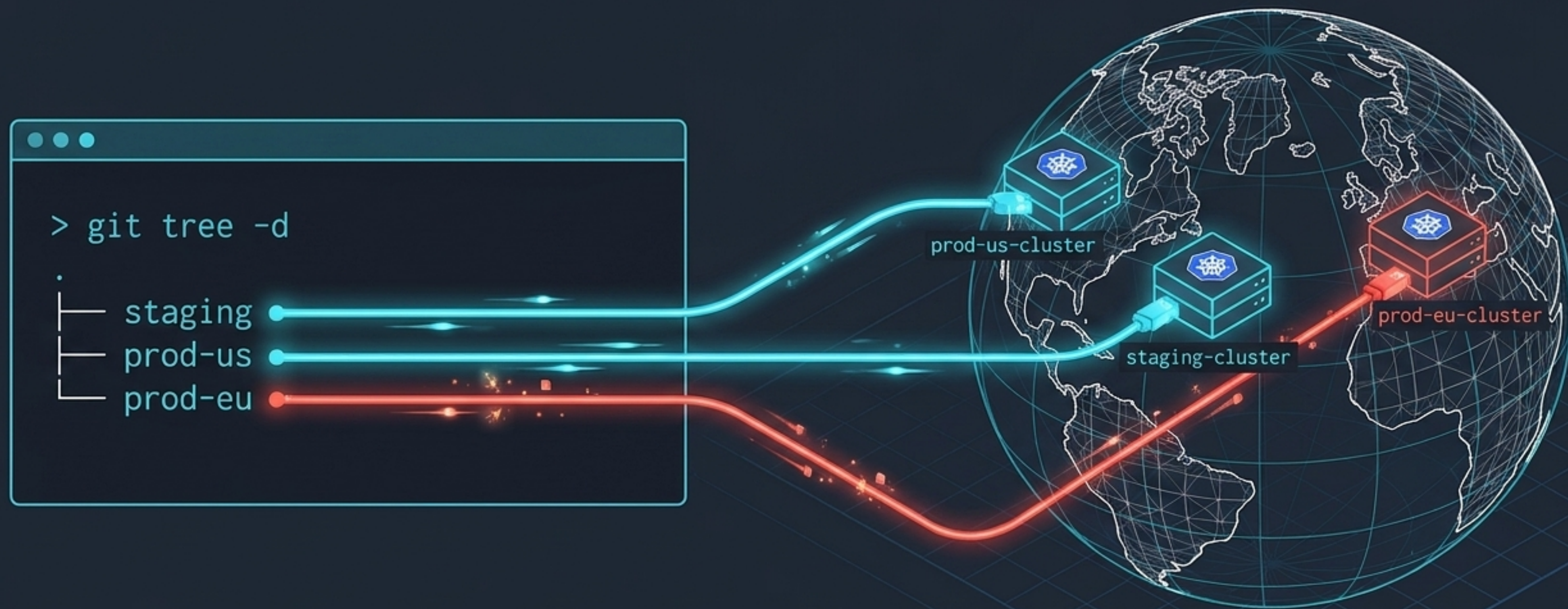
External secret operators bridge the gap between enterprise vaults and GitOps declarative state.

Progressive Delivery: Argo Rollouts



- **Beyond Basic Deployments:** Native Kubernetes limits deployments to basic rolling updates.
- **Canaries & Blue/Green:** Argo Rollouts enables precise traffic shaping based on actual user metrics.
- **Automated Rollbacks:** If the analysis engine detects latency or errors in the Canary, the GitOps loop instantly halts the rollout and routes 100% of traffic back to Stable.

Multicloud Scale: Argo CD ApplicationSets



The Git Generator

ApplicationSets use a template-driven approach to map Git directory structures directly to physical clusters.

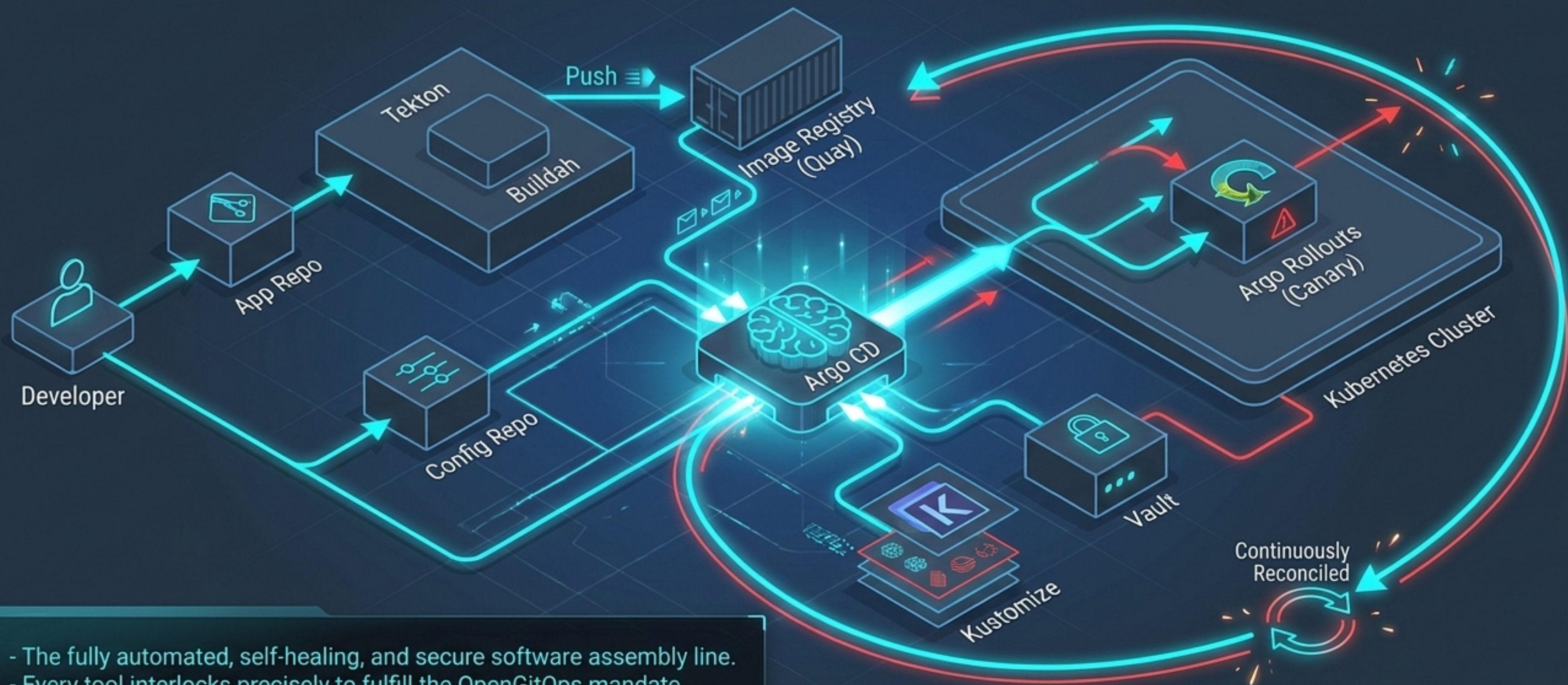
Zero-Touch Provisioning

To spin up a brand new environment, a developer simply adds a new directory in the Git repository.

Macro-Management

Deploy and configure hundreds of clusters from a single, centralized manifest repository.

Synthesis: The Complete GitOps Blueprint



- The fully automated, self-healing, and secure software assembly line.
- Every tool interlocks precisely to fulfill the OpenGitOps mandate.
- No manual interventions. Total auditability.

The GitOps Promise



Standardized Workflows:

Developers operate infrastructure using the same Git pull requests they use for code.



Enhanced Security:

Drift detection instantly neutralizes unauthorized manual changes.



Total Visibility:

The Git history becomes a flawless, compliant audit trail of every cluster state.



Reduced Lead Time:

Teaching the lumbering enterprise elephant to dance through absolute, declarative automation.